L1A: Introduction; Elements of Programming

CS1101S: Programming Methodology

Boyd Anderson

12th August, 2026

Version 1.1 — Built: 2026-08-14 21:22:13

What is CS1101S?

Course management

Elements of programming

The goals of this course

- Understand
  the *structure and interpretation of computer programs*
- Learn how to program
- Learn computational thinking
- Get glimpses into the subject areas of computer science
- Get a feeling for how computer science "ticks"
- Fall in love with computer science

Course origins

Course concept developed at MIT in the 1980s,
by Hal Abelson and Jerry Sussman

- Book: "Structure and Interpretation of Computer Programs"
- A.K.A. the "SICP"

CS1101S Legacy: The Beginning

Introduced in DISCS in Sem 1 1997/1998 as "IC1101S"
Lecturers: Jacob Katzenelson and Leong Tze Yun

Jacob renamed it to CS1101S in 1998/1999 Sem 1

CS1101S Today

- Martin Henz took over the course in 2012
- Adapted SICP for JavaScript instead of Scheme
- Low Kok Lim joined in 2014. He teaches the semester two version of the course!
- Boyd joined in 2020 and Sanka joined in 2022.
- In 2026, we moved from JavaScript to Python.

The weekly flow

- Wednesday 2-h **L**ecture XA: introduces new concepts, includes a Check-**I**n
- Wednesday **P**ath: online post-lecture quiz in Source Academy
- Thursday 1-h **R**eflection: reflects on lecture material
- Friday 2-h **L**ecture XB: introduces new concepts, includes a Check-**I**n
- Friday **P**ath: online post-lecture quiz in Source Academy
- Monday/Tuesday 2-h **S**tudio session: **Master the concepts!**
- **M**issions, **Q**uests, **C**ontests: Show your mastery!

Lecture venues

We have two venues for lectures

- Canvas Lecture Section L1: LT16
- Canvas Lecture Section L2: LT38

Please note

Cadets, report to **your** allocated venue and only your allocated venue. The teaching staff will alternate between the two venues to ensure that all cadets get a live lecture experience.

Homework to be submitted throughout the semester

Throughout the semester, we will have...

- 17 Paths
- 20 Missions
- 15 Quests

(some adjustment possible)

This week

- today 2-h **L**ecture L1A: Elements of Programming
- today Check-**I**n I1A on Elements of Programming
- today **P**ath P1A on Elements of Programming
- tomorrow 1-h **R**eflection R1: 10am and 2pm sessions
- Friday 2-h **L**ecture L1B: Runology
- Friday Check-**I**n I1B on Runology
- Friday **P**ath P1B on Runology
- Friday Mission "Rune Trials" comes out, due on Tuesday 11:59 pm
- Monday/Tuesday 2-h **S**tudio session S2 on Elements of Programming

Lecturers

Their roles

Conduct **lectures** on Wednesdays and on Fridays, coordinate the course

Martin Henz

Sanka Rasnayaka

Boyd Anderson

Tutors

Their role

Facilitate weekly **Reflection** sessions

Eldric Liew Chun Tze

Enzio Kam Hai Hong

Zhang Yao

Martin Henz, Sanka Rasnayaka, Boyd Anderson

Source Academy Team

Their roles

Design, develop, deploy, maintain the **Source Academy**, our learning environment for CS1101S

Zhang Yao, Source Academy CTO

Vemulapalli Akshay: Backend

Jain Shrey, Malani Aarav Sajeel: Frontend, Plugins, Modules, Evaluators

Staff Photos

The team

Avengers

Their role

- facilitating weekly 2-h Studio sessions,
- guiding you in the fine art of programming,
- discussing your missions and quests with you and grading them
- conducting mastery checks (more on this later)

Course Overview

- Unit 1—Functions (textbook Chapter 1)
  - Getting acquainted with the elements of programming, using **functional abstraction**
  - Learning to read programs, and using the **substitution model**
  - Example applications: Runes, curves
- Unit 2—Data (textbook Chapter 2)
  - Getting familiar with data: pairs, linked lists, trees, streams
  - Searching in linked lists and trees, sorting of linked lists
  - Example application: sound processing

Course Overview, continued

- Unit 3—State (parts of textbook Chapter 3)
  - Programming with stateful abstractions
  - Lists, loops, searching in and sorting of lists
  - Reading programs using the environment model / CSE machine
  - Example applications: robotics, video processing
- Unit 4—Beyond (parts of textbook Chapters 3 and 4)
  - Memoization, Memoized Streams
  - Metalinguistic abstraction

Special Event: Sumobot

Wednesday, October 14th, 2026. Details to come.

Assessment timeline

- **R**eading **A**ssessment RA1 on Friday Week 4: 4/9 (Unit 1)
- **P**rogramming **A**ssessment PA1 on Friday Week 6: 18/9 (Units 1 and 2)
- **Recess Week**
- **Midterm** Assessment on Wednesday Week 7: 30/9 (Unit 2)
- **R**eading **A**ssessment RA2 on Friday Week 10: 23/10 (Unit 3)
- **P**rogramming **A**ssessment PA2 on Friday Week 12: 6/11 (Unit 4)
- **Final** Assessment: 25/11

Assessment components (1/2)

5%: Source Academy XP (Missions, Quests, Paths, Studios, ...): 24,000 XP

5%: Studio: attendance 3%, effort 2%

5%: Reflection attendance

3%: Check-In (best 18 of 20)

Assessment components (2/2)

8%: Reading Assessment 1, Week 4

8%: Programming Assessment 1, Week 6

4%: Mastery Check 1, any time

20%: Midterm Assessment, Week 7

8%: Reading Assessment 2, Week 10

8%: Programming Assessment 2, Week 12

4%: Mastery Check 2, any time

30%: Final Assessment

(Wait! That's 108%...)

Best 3 of 4 for RA/PA!

5%: based on XP in Source Academy

Your homework

Homework (Paths, Missions, Quests, Studios) earn XP.

Early submission bonus

Submit Missions and Quests within 3 days of release: 75 XP bonus. Submit Paths within 1 day of release: 25 XP bonus.

Contests

Look out for announcements. Winners & runners-up receive XP. You will also earn XP for participating in the contest voting.

Miscellaneous special topics

Extra XP to deserving individuals; look out for announcements.

Resources

Textbook: SicPy

Text Book can be found here: SicPy

Course Schedule and Important Links

Canvas > CS1101S

Course FAQs can be found here

Canvas > CS1101S > Frequently Asked Questions

Need help?

- Ed Discussion forums:
  `https://edstem.org/us/courses/101145/discussion`
  Many of you are signed up, already. If you cannot sign up, please email Martin
- PM your Avenger (WhatsApp/FB/Telegram/you-name-it: check what channel they prefer)

Need help?

- Email your Reflection instructor:
  Canvas > CS1101S > Teaching Staff
- Email lecturers
  - Martin: henz@nus.edu.sg
  - Sanka: sanka@nus.edu.sg
  - Boyd: boyd@nus.edu.sg

Academic Honesty

The University takes a strict view of cheating in any form. Any student who is found to have engaged in such misconduct will be subjected to disciplinary action by the University.

Plagiarism

The practice of taking someone else's work or ideas and passing them off as one's own (e.g. copying from your classmates, seniors, books and/or online resources).

Be sure to read the Code of Student Conduct

AI and You

- You have probably already used them for fun or for your work. The teaching team is definitely using them to improve the course and Source Academy.
- These AI tools will be a great resource for you in your learning journey. Make sure you try them out. Ask your Avenger how they are using them.
- Good news: These AI tools know our material very well. There might still be mistakes in their answers, so make sure you verify the answers by asking your Avenger, tutor, or lecturer.

Our AI Policy

- We strongly discourage AI tools for Missions/Quests. We need you to use your own understanding to solve our assignments. This practice is important.
- You may use **AI tools with attribution** for Missions/Quests. However, don't take shortcuts!
- Importantly, you won't be able to use AI tools in our sit-in assessments, mastery checks or in our Studios.
- If we use them to just get an answer, we are cheating ourselves out of a learning opportunity.
- You can however use them for preparation for your reflection session. Coming up tomorrow!

Elements of programming

- Processes and programming
- Primitive expressions, 1.1.1
- Operator combinations, 1.1.1
- Evaluating combinations, 1.1.3
- Naming abstraction, 1.1.2
- Functional abstraction, 1.1.4
- Predicates and conditional expressions, 1.1.6

Processes

Definition (from Wikipedia)

A process is a set of activities that interact to produce a result. The activities unfolds according to patterns that *de*scribe or *pre*scribe the process.

Examples

Processes are everywhere. They permeate our nature and culture:

- Galaxies and solar systems follow *formation* processes.
- Metabolic pathways in our bodies are *biological* processes.
- Political parties, legislature, courts, etc. follow processes.
- Industrial production is governed by processes.

Computational processes

Definition

A *computational process* is a set of activities in a computer, designed to achieve a desired result.

Our task

Here we are concerned about how this *design* happens.

Our design method

We use *programs* to prescribe how the computational process unfolds.

What is programming?

People in focus

As the complexity of computer systems increases, *communication* between architects, designers, programmers, operators and users becomes increasingly important.

Programs as communication devices

Since programs prescribe the computational processes at the heart of these systems, they can play a central role in the *human* communication during their construction and operation.

Our motto

Programming is communicating computational processes.

Our programming environment: Source Academy

Website designed for CS1101S

Has just what you need for understanding the *structure and interpretation of computer programs*.

Research

Source Academy is also an *educational research tool*: We want to study what happens when people like you learn how to program.

Your first login to Source Academy

Consent

You can be part of our research by letting us use your programs anonymously. But if you don't want to: No worries: There will be **no penalty** if you don't consent.

Our ship

You think you are studying at NUS? Wrong! You are cadets in... the Source Academy!

Visit `https://sourceacademy.nus.edu.sg`, decide on consent and click on "Playground". (Source Academy opens later in the lecture.)

Elements of programming

Every powerful language provides...

- Primitives
- Combination
- Means of abstraction

Primitive expressions

- Numerals: 0, -42, 486
- Numerals use decimal notation
- Our interpreter can evaluate numerals, resulting in the numbers they represent

A detail

Evaluating an expression alone shows nothing

If you give the interpreter just 486, it responds by printing — nothing.

Example

To see the result, apply the primitive function `print` to it: `print(486)`

Means of Combination: Operators

Examples

```
print(5 * 99)
```

```
print(25 - (4 + 2) * 3)
```

Notation as usual

operator between operands: *infix notation* with *precedences*

Evaluating Operator Combinations

But exactly how...

...do we evaluate

```
print((2 + 4 * 6) * (3 + 12))
```

Evaluation of expressions

Demonstration: Evaluation of expressions

Evaluation of expressions

```
1 + 2 * 3 + 4
```

Replace “eligible” subexpression with result

```
(1 + (2 * 3)) + 4

(1 + 6) + 4

7 + 4

11
```

Means of Abstraction: Naming

Example

```
size = 2
print(5 * size)
```

`size = 2` is a *declaration assignment*: the name `size` is declared and identified with the value `2`.

Evaluation of declaration assignments

```
a = 3 * 5
print(a + 5)
```

Evaluate what is after the $=$

```
size = 3
2 * size
```

Then replace that name in rest of the program with that value.

```
2 * 3
```

```
6
```

Primitive Names

Primitive constants

Python has a few primitive names. For example, the name `math_pi` refers to the constant π.

Primitive functions

In addition to operators such as +, Python has primitive functions. For example, `math_sqrt` is the square root function.

Function application expressions

Functions can be applied as usual in mathematics, for example

```
print(math_sqrt(15))
```

applies the primitive square root function to 15.

Means of Abstraction: Compound functions

Example

```
def square(x):
    return x * x
```

What does this statement mean?

Like declaration assignments, this *function definition* declares a name, here `square`. The value associated with the name is a function that takes an argument `x` and produces (returns) the result of multiplying `x` by itself.

Formatting matters: Indentation

Python uses indentation

Python uses *indentation* to show which statements belong to a function's body.

Correctly indented

```
def square(x):
    return x * x

print(square(5))
```

Formatting matters: Indentation (continued)

Incorrectly indented

```
def square(x):
return x * x
```

Python can't tell that `return x * x` belongs inside `square` — so it refuses to guess, and raises an error instead.

Review: Function application

Recall from the `math_sqrt` example

We apply a function by supplying its arguments in parentheses.

Example

```
def square(x):
    return x * x

print(square(14))
```

More examples

```
print(square(21))

print(square(2 + 5))

print(square(square(3)))
```

Boolean values

Boolean values

The two boolean values `True` and `False` represent *answers* to yes-no questions

Predicates

A *predicate* is a function or an expression that returns or evaluates to a boolean value.

Examples

```
True
False
x >= 1
```

Another predicate

Example

```
def adult(x):
    return x >= 18
```

Conditional expressions

Why?

We would like to check something, e.g. answer a yes/no question, and return a different value, depending on the answer

Back to the example

```
enter_club() if adult(my_age) else go_to_movie()
```

Example: `abs` function

To compute the absolute value of a number, check if it is greater or equal 0. If yes, return the number unchanged, and if no, return the number negated.

Time for the Check-In!

Still here? Time to head to the Source Academy and check in!

Look for our first Check-In in the Source Academy!

May the Source be with you!

The Source Academy is open!

Look out for your first Path “Elements of Programming”.

Welcome on board!